PROJECT JARVIS • TACTICAL ENGINEERING MANUAL

3-Day Deep-Dive Execution Playbook

A zero-to-one guide for 4 engineers building an autonomous desktop AI assistant with Antigravity

---

Sprint Timeline: Days 1 to 3 (Starting Tomorrow)	**Target Stack:** Python 3.11, Antigravity, FastAPI, Gemini API
Target Deliverable: Working ReAct Desktop Agent MVP	**Governance Model:** Modular Isolation & Git Feature Branches

Role & Engineer	Folder Territory	3-Day Focus & Deliverables
Jay Lead & Core Brain	`core/` *(Only Jay edits)*	Repository initialization, LLM connectivity (Gemini API), multi-turn conversation memory, and autonomous function-calling (ReAct) execution loop.
Aryan Voice Engineer	`voice/` *(Only Aryan edits)*	Audio hardware validation (16kHz mono), Speech-to-Text (STT) transcription pipeline, and low-latency Neural Text-to-Speech (TTS) synthesis.
Abhijit System & Automation	`tools/` *(Only Abhijit edits)*	OS telemetry tools (CPU, RAM, battery), asynchronous application launcher, sandboxed file operations, and security confirmation guardrails.
Rohit Frontend & Memory	`memory/ & ui/` *(Only Rohit edits)*	FastAPI backend server, persistent SQLite notes and interaction logs, real-time WebSocket bridge, and sci-fi dark mode HUD chat interface.

The Learning & Engineering Philosophy: You are using Antigravity as an accelerator, not a magic crutch. Every engineer must understand their code line-by-line. Never accept generated code you cannot explain in a technical viva or interview. To avoid merge disasters, everyone adheres strictly to their assigned module directory. If you need a function from another module, request that teammate to expose an interface—**never edit their folder directly**.

CHAPTER 1: UNIVERSAL SETUP & GIT PROTOCOL

Every teammate must follow this setup sequentially before beginning Day 1 tasks.

1.1 Git & GitHub Configuration (Everyone)

1. **Create Account:** Sign up at github.com if you do not have an account.
2. **Install Git:** Download and install *Git for Windows* from git-scm.com. Ensure **Git Credential Manager** is checked.
3. **Configure Identity:** Run in your PowerShell:

```
git config --global user.name "Your Full Name"
git config --global user.email "your_email@example.com"
```

1.2 Repository Setup & Branching Strategy

• **Jay** initializes the repository on GitHub (jarvis-desktop-assistant) and invites Aryan, Abhijit, and Rohit as collaborators.
• **Aryan, Abhijit, and Rohit** accept the GitHub email/notification invite, then clone the repository onto their laptops:
git clone https://github.com/JAY_USERNAME/jarvis-desktop-assistant.git
• **Branching Rule:** NEVER commit directly to main. Each developer works in a dedicated branch:

```
# Jay's branch:
git checkout -b feature/jay-core

# Aryan's branch:
git checkout -b feature/aryan-voice

# Abhijit's branch:
git checkout -b feature/abhijit-tools

# Rohit's branch:
git checkout -b feature/rohit-ui-memory
```

1.3 Virtual Environment & Secrets Protection

Run these commands in PowerShell in the project root:
1. python -m venv venv
2. .\venv\Scripts\Activate.ps1 (If permission error: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass)
3. pip install -r requirements.txt
4. cp .env.example .env (Add your API key inside .env. **NEVER COMMIT .ENV!**)

1.4 How to Open and Prompt Antigravity Correctly

■■ **WHICH FOLDER TO OPEN:** Open the **ROOT project folder** (Project - A) in Antigravity. **NEVER** open a single subfolder (like voice/) alone, because Antigravity will lose sight of ARCHITECTURE.md, AGENTS.md, requirements.txt, and root Python imports.

The Golden Antigravity Prompting Rule: Every prompt must start with your identity and territory constraint:

> *"I am [Your Name], the [Your Role] for Project Jarvis. You are ONLY allowed to create or edit files inside the [your_folder]/ directory. Do not touch any other folder, ARCHITECTURE.md, or .env. Use Python 3.11 with strict type hints and docstrings. Wrap all logic in try/except and provide a runnable verification test script."*

1.5 The End-of-Day Git Cadence (Push & Pull Request)

At the end of each day or feature, stage only your folder, commit, and push to GitHub:

```
git status # Verify .env is NOT listed under untracked files!
git add [your_folder]/ # e.g., git add voice/
git commit -m "feat: implement day X milestone"
git push -u origin feature/[your-branch]
# Then open GitHub in browser and click 'Compare & Pull Request' targeting main
```

1.6 How to Sync Latest Merged Code From Main

Whenever Jay merges a pull request on GitHub, update your local branch cleanly without conflicts:

```
# Step 1: Switch to local main and pull newest commits
git checkout main
git pull origin main

# Step 2: Switch back to your feature branch and merge main into it
git checkout feature/[your-branch]
git merge main
```

CHAPTER 2: JAY • TEAM LEAD & CORE BRAIN

Ownership Territory: core/, ARCHITECTURE.md, central runners (main_cli.py), merging PRs.

■ Day 1: Central Repository Initialization & LLM API Connectivity

Today's Goal: Launch the central GitHub repo, invite teammates, establish contracts, and verify LLM API calls in Python.

How to Do It (Step-by-Step):
1. Run git init, set remote, push initial commit with ARCHITECTURE.md, AGENTS.md, .gitignore.
2. Add Aryan, Abhijit, and Rohit as collaborators on GitHub.
3. Obtain Gemini API key from Google AI Studio. Add to .env as GEMINI_API_KEY.
4. Create core/test_llm.py using google-genai to call gemini-2.5-flash.

■ EXACT ANTIGRAVITY PROMPT: Jay Day 1 — LLM Sanity Check

```
I am Jay, Lead of Project Jarvis. You may ONLY create and edit files inside core/.
Read .env to retrieve GEMINI_API_KEY using python-dotenv. Use the google-genai SDK.
Create core/test_llm.py that initializes the Gemini client with model 'gemini-2.5-flash'.
Send a prompt: 'Hello, confirm your identity as Jarvis Core.'
Print the response with elapsed latency in seconds. Handle missing API key exceptions cleanly.
Include strict type hints and docstrings.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git checkout -b feature/jay-core
git add core/test_llm.py
git commit -m "feat(core): initialize LLM test runner with gemini-2.5-flash"
git push -u origin feature/jay-core
```

Acceptance Test & Expected Output: Run python core/test_llm.py. Terminal prints: *"Jarvis core initialized successfully. Latency: 0.42s"*.

■ Day 2: Multi-Turn Conversation Memory & System Prompt Persona

Today's Goal: Build core/brain.py with multi-turn conversation memory and system prompt persona.

How to Do It (Step-by-Step):
1. Create class JarvisBrain in core/brain.py.
2. Configure System Prompt: Concise, witty, tech-savvy assistant designed for desktop automation.
3. Implement chat(user_message: str) -> str maintaining a message history buffer.
4. Build main_cli.py at project root to run an interactive terminal chat session.

■ EXACT ANTIGRAVITY PROMPT: Jay Day 2 — Conversational Brain & CLI Loop

```
I am Jay. You may ONLY modify files in core/ and main_cli.py.
Implement JarvisBrain in core/brain.py:
1. Load GEMINI_API_KEY from .env.
2. Define system prompt: 'You are Jarvis, an ultra-efficient desktop assistant. Speak concisely in 1-2 s
entences suitable for voice.'
3. Maintain conversation history list. Add chat(message: str) -> str method.
4. Create main_cli.py with an interactive while loop: user inputs prompt, Jarvis responds. Type 'exit' t
o quit.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add core/brain.py main_cli.py
git commit -m "feat(core): implement multi-turn chat memory and terminal CLI loop"
git push origin feature/jay-core
```

Acceptance Test & Expected Output: Run python main_cli.py. Tell it: *"My name is Jay"*, then ask *"What is my name?"*. Jarvis replies: *"Your name is Jay."*.

■ Day 3: Autonomous Function-Calling (ReAct) Tool Execution Engine

Today's Goal: Enable the Brain to dynamically inspect Abhijit's tool schemas, execute them, and interpret results.

How to Do It (Step-by-Step):
1. Import tools.registry into core/brain.py.
2. Convert tool schemas into Gemini Function Declarations.
3. When the LLM emits a tool call, extract name and arguments.
4. Call tools.registry.execute_tool(name, args), capture ToolResult, and feed it back to Gemini for the final natural language response.

■ EXACT ANTIGRAVITY PROMPT: Jay Day 3 — ReAct Function Calling Loop

```
I am Jay. You may ONLY modify files inside core/.
Integrate Abhijit's tools from tools.registry into the Gemini function calling flow in core/brain.py:
1. Retrieve registered tools and convert their get_schema() into Gemini Tool declarations.
2. When sending messages, pass tools=tools.
3. If model responds with function_calls, iterate over calls, execute via tools.registry.execute_tool(na
me, args), and return ToolResult.to_llm_string().
4. Send tool output back to Gemini so it summarizes the result to the user.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add core/brain.py tests/test_brain.py
git commit -m "feat(core): implement ReAct tool-calling loop for autonomous execution"
git push origin feature/jay-core
```

Acceptance Test & Expected Output: In main_cli.py, type: *"Check my RAM and open Notepad"*. Notepad pops up, and Jarvis replies: *"I've opened Notepad, and your RAM usage is 42%."*.

CHAPTER 3: ARYAN • VOICE ENGINEER

Ownership Territory: voice/, microphone capture, STT, and neural TTS voice streaming.

■ Day 1: Audio Hardware Diagnostics & Sampling Verification

Today's Goal: Verify laptop microphone input and speaker output using Python sounddevice and numpy.

How to Do It (Step-by-Step):
1. Checkout branch: git checkout -b feature/aryan-voice.
2. Install packages: pip install sounddevice numpy soundfile.
3. Create voice/audio_check.py to list audio devices and perform a 3-second recording and playback test.

■ EXACT ANTIGRAVITY PROMPT: Aryan Day 1 — Audio Diagnostics

```
I am Aryan, Voice Engineer for Project Jarvis. You may ONLY modify files inside voice/.
Create voice/audio_check.py using sounddevice, soundfile, and numpy:
1. Query and list all audio host APIs and default input/output devices.
2. Record 3 seconds of audio from the default microphone at 16000 Hz, mono channel, 16-bit PCM.
3. Save the recording to temp_audio/mic_test.wav (create folder if missing).
4. Play the recorded audio back through default speakers.
5. Include clear error handling if mic permissions are denied.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git checkout -b feature/aryan-voice
git add voice/audio_check.py
git commit -m "feat(voice): implement 16kHz audio recording and playback check"
git push -u origin feature/aryan-voice
```

Acceptance Test & Expected Output: Run python voice/audio_check.py. Speak during the 3 seconds; your voice plays back clearly through laptop speakers.

■ Day 2: Speech-to-Text (STT) Transcription Pipeline

Today's Goal: Build voice/stt.py to capture spoken commands and transcribe them into clean text.

How to Do It (Step-by-Step):
1. Select STT engine: faster-whisper (offline CPU) or Groq Whisper API (instant cloud).
2. Implement Transcriber class with transcribe_mic(duration=4) -> str.
3. Build voice/test_stt.py to verify accuracy on speech.

■ EXACT ANTIGRAVITY PROMPT: Aryan Day 2 — STT Transcriber

```
I am Aryan. You may ONLY modify files inside voice/.
Create voice/stt.py implementing a Speech-to-Text transcriber.
Use faster-whisper (tiny or base model) for local transcription.
Signature: transcribe_mic(duration_seconds: int = 4) -> str.
1. Record audio at 16kHz mono.
2. Run Whisper model inference on audio buffer.
3. Return normalized, stripped text string.
Create voice/test_stt.py: prompts 'Speak now...', records, and prints 'Transcript: [your text]'.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add voice/stt.py voice/test_stt.py
git commit -m "feat(voice): implement speech-to-text pipeline using whisper"
git push origin feature/aryan-voice
```

Acceptance Test & Expected Output: Run python voice/test_stt.py. Say: *"Jarvis what is the date today?"*. Terminal outputs matching text with >90% accuracy.

■ Day 3: Neural Text-to-Speech (TTS) Voice Engine

Today's Goal: Implement high-fidelity neural speech generation so Jarvis can speak back naturally.

How to Do It (Step-by-Step):

1. Install edge-tts and pygame (or playsound3).
2. Create voice/tts.py with JarvisSpeaker class.
3. Configure voice model: en-US-GuyNeural.
4. Implement speak(text: str) with non-blocking audio playback.

■ EXACT ANTIGRAVITY PROMPT: Aryan Day 3 — Neural TTS Synthesis

```
I am Aryan. You may ONLY modify files inside voice/.
Create voice/tts.py using edge-tts and sounddevice/playsound3.
1. Define JarvisSpeaker with default voice 'en-US-GuyNeural'.
2. Method async def speak_async(text: str): stream audio from edge-tts to temp_audio/speech.mp3 and play
 it.
3. Method def speak(text: str): synchronous helper for easy calling.
4. Handle empty strings, special characters, and audio driver busy locks gracefully.
Create voice/test_tts.py that speaks: 'Good evening Jay. All systems are operational.'
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add voice/tts.py voice/test_tts.py
git commit -m "feat(voice): implement edge-tts neural voice generation engine"
git push origin feature/aryan-voice
```

Acceptance Test & Expected Output: Run python voice/test_tts.py. Laptop speakers speak the phrase in a crisp, natural human voice in under 1 second.

CHAPTER 4: ABHIJIT • SYSTEM & AUTOMATION ENGINEER

Ownership Territory: tools/, OS automation, app launching, file sandboxing, safety guardrails.

■ Day 1: OS Telemetry & Asynchronous Process Spawning

Today's Goal: Build raw OS telemetry probes and verify asynchronous application launching on Windows.

How to Do It (Step-by-Step):
1. Checkout branch: git checkout -b feature/abhijit-tools.
2. Review tools/base.py to internalize BaseTool and ToolResult.
3. Install packages: pip install psutil pyautogui pillow.
4. Create tools/test_system.py to test CPU, RAM, battery sensors and non-blocking subprocess.Popen.

■ EXACT ANTIGRAVITY PROMPT: Abhijit Day 1 — OS Telemetry & Process Spawning

```
I am Abhijit, System Engineer for Project Jarvis. You may ONLY modify files inside tools/.
Study tools/base.py. Never modify tools/base.py or ARCHITECTURE.md.
Create tools/test_system.py using psutil and subprocess:
1. Extract CPU % (interval=0.2), used and total RAM in GB, and battery percentage + plugged status.
2. Launch notepad.exe using subprocess.Popen (ensure non-blocking, so python script continues immediatel
y).
3. Print telemetry in structured JSON.
4. Ensure it runs cleanly on Windows 10/11.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git checkout -b feature/abhijit-tools
git add tools/test_system.py
git commit -m "feat(tools): implement system telemetry probe and non-blocking app launch"
git push -u origin feature/abhijit-tools
```

Acceptance Test & Expected Output: Run python tools/test_system.py. Notepad opens immediately, and CPU/RAM JSON stats print in terminal.

■ Day 2: Standardized System Tools & Central Tool Registry

Today's Goal: Implement core tools adhering to BaseTool and construct tools/registry.py.

How to Do It (Step-by-Step):
1. Implement GetSystemStatsTool, OpenAppTool, and ListDirectoryTool in tools/system_tools.py.
2. Build tools/registry.py: Dictionary registering tools by name.
3. Implement execute_tool(name: str, args: dict) -> ToolResult with robust exception handling.

■ EXACT ANTIGRAVITY PROMPT: Abhijit Day 2 — System Tools & Registry

```
I am Abhijit. You may ONLY modify files inside tools/.
Implement the following tools inheriting from BaseTool in tools/base.py:
1. OpenAppTool in tools/system_tools.py: takes app_name: str. Maps shortcuts ('chrome', 'code', 'calc',
'notepad', 'spotify') to Windows executable paths. Returns ToolResult.
2. ListDirectoryTool in tools/system_tools.py: takes directory_path: str (defaults to '.'). Returns list
 of files/folders.
3. Create tools/registry.py: dictionary registering GetSystemStatsTool, OpenAppTool, and ListDirectoryTo
ol. Function execute_tool(name: str, args: dict) -> ToolResult that executes safely and catches unknown
tool errors.
Include get_schema() returning valid JSON schemas for LLM tool calling.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add tools/system_tools.py tools/registry.py
git commit -m "feat(tools): implement standardized system tools and central execution registry"
git push origin feature/abhijit-tools
```

Acceptance Test & Expected Output: Run python -c "from tools.registry import execute_tool; print(execute_tool('open_app', {'app_name': 'calc'}))". Calculator opens and ToolResult prints success=True.

■ Day 3: Sandboxed File Operations & Safety Guardrail Engine

Today's Goal: Create safe file reading/writing tools with sandbox guardrails preventing accidental data destruction.

How to Do It (Step-by-Step):
1. Create tools/safety.py: Restrict file operations to safe folders; block root system paths (C:\Windows, C:\Program Files).
2. Create tools/file_tools.py: ReadFileTool, WriteFileTool.
3. Implement overwrite confirmation: if a file exists and overwrite=False, require confirmation.

■ EXACT ANTIGRAVITY PROMPT: Abhijit Day 3 — Sandboxed File Tools & Safety Guards

```
I am Abhijit. You may ONLY modify files inside tools/.
1. Create tools/safety.py with is_path_safe(path: str) -> bool. Disallow paths inside Windows system dir
ectories and enforce sandbox restrictions.
2. In tools/file_tools.py, implement ReadFileTool(file_path: str) -> ToolResult.
3. In tools/file_tools.py, implement WriteFileTool(file_path: str, content: str, overwrite: bool = False
) -> ToolResult. If file exists and overwrite is False, return success=False asking for confirmation.
4. Register both in tools/registry.py.
Write unit tests in tests/test_tools.py checking both successful writes and blocked dangerous paths.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add tools/file_tools.py tools/safety.py tests/test_tools.py
git commit -m "feat(tools): implement sandboxed file tools and safety guardrail engine"
git push origin feature/abhijit-tools
```

Acceptance Test & Expected Output: Run pytest tests/test_tools.py. Sandbox file write succeeds; writing into C:\Windows\System32 returns Permission Denied.

CHAPTER 5: ROHIT • FRONTEND & MEMORY ENGINEER

Ownership Territory: memory/, ui/, SQLite database, FastAPI REST/WebSocket server, Web HUD.

■ Day 1: FastAPI Backend Setup & Persistent SQLite Database

Today's Goal: Set up the backend server and initialize the SQLite schema for persistent notes and interaction logs.

How to Do It (Step-by-Step):

1. Checkout branch: git checkout -b feature/rohit-ui-memory.
2. Install packages: pip install fastapi uvicorn[standard] aiosqlite websockets.
3. Create memory/db.py to initialize memory/jarvis.db with tables: notes and chat_history.
4. Create ui/server.py with FastAPI application and healthcheck endpoint.

■ EXACT ANTIGRAVITY PROMPT: Rohit Day 1 — Backend & SQLite Database

```
I am Rohit, Frontend & Memory Engineer for Project Jarvis. You may ONLY modify memory/ and ui/.
1. In memory/db.py, write async functions to initialize memory/jarvis.db with aiosqlite:
   - Table notes: id INTEGER PRIMARY KEY, title TEXT, content TEXT, created_at TIMESTAMP
   - Table chat_history: id INTEGER PRIMARY KEY, role TEXT, content TEXT, timestamp TIMESTAMP
   - Enable WAL mode (PRAGMA journal_mode=WAL) for high performance.
2. In ui/server.py, create a FastAPI app. Include an @app.on_event('startup') handler that calls db init
ialization.
3. Add GET /health returning {'status': 'healthy', 'db': 'connected'}.
Include unit test in tests/test_memory.py.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git checkout -b feature/rohit-ui-memory
git add memory/db.py ui/server.py tests/test_memory.py
git commit -m "feat(backend): initialize FastAPI server and SQLite persistent store"
git push -u origin feature/rohit-ui-memory
```

Acceptance Test & Expected Output: Run uvicorn ui.server:app --reload. Visit http://127.0.0.1:8000/health. JSON returns: {"status": "healthy"}.

■ Day 2: Note Taking CRUD & REST API Endpoints

Today's Goal: Build full CRUD operations for long-term memory notes and expose REST endpoints for the assistant.

How to Do It (Step-by-Step):
1. Create memory/storage.py: create_note(title, content), get_notes(), search_notes(keyword).
2. Expose in ui/server.py: POST /api/notes (Pydantic validated) and GET /api/notes.
3. Add endpoint GET /api/telemetry calling Abhijit's GetSystemStatsTool to verify module interop.

■ EXACT ANTIGRAVITY PROMPT: Rohit Day 2 — Memory Manager & REST APIs

```
I am Rohit. You may ONLY modify memory/ and ui/.
1. Create memory/storage.py with class MemoryManager:
   - async add_note(title: str, content: str) -> int
   - async list_notes(limit: int = 20) -> list[dict]
   - async search_notes(query: str) -> list[dict]
2. In ui/server.py, define Pydantic schema NoteCreate(title: str, content: str).
3. Add POST /api/notes (creates note) and GET /api/notes (retrieves notes).
4. Add GET /api/telemetry which calls tools.system_tools.GetSystemStatsTool().run() to display live syst
em stats on API.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add memory/storage.py ui/server.py
git commit -m "feat(memory): implement note taking CRUD and REST API endpoints"
git push origin feature/rohit-ui-memory
```

Acceptance Test & Expected Output: Open http://127.0.0.1:8000/docs. Use interactive Swagger UI to POST note 'Buy Groceries'. Verify it returns 201 Created and persists in SQLite.

■ Day 3: WebSocket Live Channel & Sci-Fi Web HUD Client

Today's Goal: Build the real-time WebSocket channel and a sleek dark-mode Jarvis HUD chat interface.

How to Do It (Step-by-Step):
1. Add WebSocket route @app.websocket('/ws/jarvis') in ui/server.py.
2. Build single-page web client ui/index.html styled with dark sci-fi aesthetic (`#0f172a`, cyan glowing borders, monospace font).
3. Implement live message exchange: user sends prompt -> UI displays 'Thinking...' badge -> receives streamed chunks or tool status.

■ EXACT ANTIGRAVITY PROMPT: Rohit Day 3 — WebSockets & Sci-Fi HUD Client

```
I am Rohit. You may ONLY modify memory/ and ui/.
1. In ui/server.py, add @app.websocket('/ws/jarvis'). Handle incoming JSON: {'action': 'chat', 'text': s
tr}.
2. Push progress events: {'type': 'status', 'status': 'thinking'}, then {'type': 'response', 'text': str
}.
3. Create ui/index.html with a futuristic Jarvis HUD interface:
   - Dark slate theme (#0f172a), cyan glowing accents, telemetry widget showing CPU/RAM.
   - Message feed displaying user and assistant chat bubbles.
   - Status indicator (Idle, Listening, Thinking, Executing Tool).
4. Mount ui/ static directory in server.py so visiting http://127.0.0.1:8000 serves the HUD.
```

■ GIT COMMANDS (RUN IN TERMINAL):

```
git add ui/server.py ui/index.html
git commit -m "feat(ui): implement real-time WebSocket channel and futuristic web HUD"
git push origin feature/rohit-ui-memory
```

Acceptance Test & Expected Output: Open http://127.0.0.1:8000 in browser. Type a message in the HUD. The UI connects over WebSocket and displays real-time responses.

CHAPTER 6: DAY 4 INTEGRATION PREVIEW

How the 4 modules lock together into a single functioning desktop assistant.

6.1 The 5-Step Integration Pipeline

Step	Module Owner	Action & Interface Contract
1. Input	Aryan (voice/) or Rohit (ui/)	Captures spoken voice via STT or typed command via WebSocket HUD; passes text string to Jay's Brain.
2. Decision	Jay (core/)	LLM evaluates prompt with memory context; determines whether to answer directly or invoke a registered tool.
3. Execution	Abhijit (tools/)	Tool registry executes OS command, app launch, or file operation; returns structured ToolResult.
4. Storage	Rohit (memory/)	Conversation turns and new user notes are saved into SQLite database.
5. Output	Aryan (voice/) & Rohit (ui/)	TTS synthesizes voice reply through speakers; WebSocket pushes live text and status update to HUD.

6.2 Daily Standup Meeting Agenda (15 Minutes Every Evening)

Hold a 15-minute voice call every evening at 8:00 PM. Each person answers strictly 3 questions:

1. **What did I build and verify today?** (Share screen for 1 minute showing your code running).
2. **What technical blocker or error did I encounter?** (Discuss unexpected bugs or API issues).
3. **Is my branch clean and pushed to GitHub for PR review?** (Jay confirms review status).

6.3 Master Troubleshooting & Antigravity Gotchas

Issue / Error	Root Cause	How to Fix It
PowerShell script execution disabled	Windows ExecutionPolicy security restriction	`Run: Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
Antigravity edits other people's folders	Prompt didn't enforce boundary rule	Prepend: *'You are ONLY allowed to edit files in [module]/'* to every prompt.
Audio device busy / crash	Multiple processes holding microphone stream	Close other recording apps; use context manager with sd.InputStream()
FastAPI freezes on tool call	Synchronous blocking call inside async route	`Wrap execution with asyncio.to_thread(execute_tool, ...)`